

Analyse v9 → v10 — W=8 forcé sur i7-7500U

Date : 2026-06-12

Branche : Riemann_Lab_C

Auteur : hprzeta

1. Problèmes de v9 (Brent C/mpfr)

1.1 Bottleneck mémoire — gain turbo décevant

v9 introduit Brent C/mpfr (IQI + sécante + bisection, ordre ~ 1.84) pour réduire les itérations d'affinage. Résultat T=100 000 :

Indicateur	v9 sans turbo	v9 avec turbo	Rapport
Durée	28.0 min	26.6 min	$\times 1.05$
Vitesse	82.15 z/s	86.5 z/s	+5 %

Le gain turbo est seulement $\times 1.05$, contre $\times 1.63$ pour v7 Illinois. La raison : Brent C/mpfr est **mémoire-bound**, pas CPU-bound. Le governor performance accélère le calcul ALU mais pas les accès MPFR (empreinte mémoire 64→80 bits par limb).

1.2 Sous-utilisation HT à grand t

Sur i7-7500U (2 cœurs physiques, 4 threads logiques HT) avec W=4 workers :

- Coût Brent à grand t : **~ 64 ms/appel** (vs ~ 10 ms Illinois v8 à petit t)
- Pendant un appel Brent à $t \approx 80\,000$, le worker est en attente d'accès mémoire MPFR
- Le thread HT associé reste idle → opportunité de parallélisation inexploitée avec W=4

Le benchmark v8 avec W=8 ($\times 0.99$) avait été mesuré avec Illinois (~ 10 ms/appel). Avec Brent à grand t (~ 64 ms/appel), le context-switch HT devient **relativement négligeable** → W=8 peut être bénéfique.

1.3 Cache (fa, fb) non implémenté

Le scan de détection calcule déjà $Z(a)$ et $Z(b)$ pour identifier le changement de signe. L'affinage Brent calcule à nouveau $Z(a)$ et $Z(b)$ lors de l'initialisation — 2 évaluations redondantes par zéro.

Phase	Évals Z	Utilité
Scan → changement de signe	2 (Z(a), Z(b))	✓ bracket
Init Brent (v9)	2 (Z(a), Z(b))	(!) redondantes
Itérations Brent	$n \times 1$	✓ convergence

2. Solution v10 — W=8 forcé

2.1 Principe

v10 ne modifie qu'**un seul paramètre** par rapport à v9 : le nombre de workers.

```
# v9
N_WORKERS = min(8, cpu_count())  # → 4 sur i7-7500U

# v10
N_WORKERS = 8                    # forcé - ignore cpu_count()
```

L'hypothèse : si chaque appel Brent coûte ~64 ms à grand t, le worker est souvent en attente → 8 workers permettent de masquer cette latence mémoire mieux que 4.

2.2 Justification théorique

Le modèle de débit avec hyperthreading :

$$\text{Débit}_{\text{HT}} = \frac{W \cdot \text{utilisation_CPU}}{t_{\text{appel}}}$$

À grand t :

- $t_{\text{appel}} = 64 \text{ ms}$ → le ratio temps_attente / t_{appel} est élevé
- HT permet à un 2ème thread de progresser pendant l'attente
- $W=8 \approx W=4 + 4 \text{ threads HT opportunistes}$ — gain réel si latence mémoire > overhead context-switch

2.3 Paramètres identiques à v9

Paramètre	v9	v10
Méthode	Brent C/mpfr	Brent C/mpfr (inchangé)
prec_fast	64 bits	64 bits
prec_full	80 bits	80 bits
STEP	0.010 fixe	0.010 fixe
ITER_SWITCH	3	3
MAX_ITER	50	50
N_WORKERS	4	8

3. Résultats T=100 000 v10 (mesurés 2026-06-12)

3.1 Validation T=10 000

Indicateur	v9 T=10k	v10 T=10k
Zéros	10 142	10 142
Manquants	0	0
Durée	16.6 s	16.2 s
Vitesse	609 z/s	624 z/s
Turing	COMPLET	COMPLET

Gain faible sur T=10k (t petit → Brent rapide → HT peu bénéfique).

3.2 Run T=100 000

Indicateur	v9 (turbo)	v10 (turbo)	Gain
Zéros	138 069	138 069	=
Manquants	0	0	=
Durée	26.6 min	23.7 min	×1.12
Vitesse	86.5 z/s	97.08 z/s	×1.12
ms/appel Brent	—	64.09 ms	—
Turing	COMPLET	COMPLET	=
LMFDB	19/20	19/20	=

Gain cumulé v1 → v10 : ×5 040 (21h → 23.7 min)

W=8 bénéfique (×1.12) malgré l'hyperthreading : le coût élevé de Brent à grand t (~64 ms/appel) dilue l'overhead de context-switch HT. Cohérent avec l'hypothèse : benchmark v8 prédisait ×0.99 pour W=8 avec Illinois (~10 ms/appel), mais Brent × 6 plus lent/appel inverse l'équilibre.

4. Tableau récapitulatif Avant/Après/Gain

Métrique	Avant (v9)	Après (v10)	Gain
Workers	4	8	+4 threads HT
Durée T=100k	26.6 min	23.7 min	-11 %
Vitesse	86.5 z/s	97.08 z/s	+12 %
ms/appel global	~11.5 ms	~10.3 ms	-10 %
Zéros manquants	0	0	=
Turing	COMPLET	COMPLET	=
Gain cumulé v1→vN	×4 500	×5 040	—

5. Limites de v10 → pourquoi v11/v12

5.1 Plafond W=8 atteint

Sur i7-7500U (2 cœurs physiques, 4 threads logiques), W=8 est la limite pratique : 2 threads par contexte HT. Au-delà de W=8, le context-switch devient contre-productif.

5.2 MPFR irréductiblement mémoire-bound

Le coût ~64 ms/appel Brent est structurel : chaque opération MPFR en 80 bits accède à la mémoire pour chaque mantisse. Aucune optimisation algorithmique sur Brent ne peut contourner cette limite physique.

5.3 Cache fa/fb toujours non implémenté

Les 2 évals Z redondantes (scan → Brent) restent présentes. Gain estimé si implémenté : ×1.1-1.3.

5.4 Conclusion : changement de backend nécessaire

Pour dépasser le plafond MPFR, il faut changer de backend de calcul Z(t) :

Backend	Coût/appel	Précision	Mode
Z_rs_mpfr (prec=64 bits)	~10 ms	~1e-9	Phase 1 Brent
Z_rs_mpfr (prec=80 bits)	~64 ms	~1e-11	Phase 2 Brent
Z_arb double natif	~0.015 ms	~2e-16	Phase 1 v12
Z_arb pleine précision	~3.5 ms	<1e-15	Phase 2 Newton (v12)

→ arb_fwrap_cdouble_hardy_z est **4 000× plus rapide** pour la phase 1. C'est le levier de v12.

6. Questions ouvertes

- Le gain W=8 (×1.12) devrait-il être proportionnel à t_{appel} ? Tester W=8 vs W=4 avec un backend plus rapide (Arb) pour vérifier.
- L'overhead HT à W=8 est-il constant ou croît-il avec le nombre de zéros concurrents ?
- Cache fa/fb : gain réel à mesurer — priorité v11 ou intégré directement en v12 ?
- Odlyzko-Schönhage : pour $T=1\,000\,000$, algorithme sous-linéaire incontournable (complexité $O(T^{1/2+\varepsilon})$).

analyse_problemes_v9_v10.md · Riemann_Lab.wiki/master · hprzeta · MAJ 2026-06-13 · ~130 lignes

